

# Git — Guia Completo do Dia a Dia

---

## O Mapa Mental do Git

```
[Seu PC]           [Git local]           [GitHub]
|                  |                      |
Edita  → git add → Stage → git commit → Repo local → git push → Remoto
                                           ← git pull ←
```

---

## 1. Configuração Inicial (uma vez só)

```
bash

git config --global user.name "Leonardo Barony"
git config --global user.email "leonardobarony@gmail.com"
git config --global core.editor "code --wait" # VS Code como editor padrão
```

---

## 2. Começar um Projeto

### Situação A — Projeto novo (começa do zero no PC)

```
bash

mkdir meu-projeto && cd meu-projeto # Cria e entra na pasta
git init                             # Inicia o Git na pasta

# Crie o repositório em github.com/new (sem README, sem .gitignore)

git remote add origin git@github.com:Leonardobarony/meu-projeto.git # Conecta
echo "# meu-projeto" >> README.md
git add .
git commit -m "feat: primeiro commit"
git push -u origin main             # Sobe pro GitHub
```

---

### Situação B — Projeto já existe no GitHub (clonar)

```
bash
```

```
# No GitHub: Code → SSH → copiar endereço
git clone git@github.com:Leonardobarony/meu-projeto.git
# Já cria a pasta, baixa tudo e conecta automaticamente ✓
```

## Verificar se está conectado ao GitHub

```
bash

git remote -v
# Deve aparecer:
# origin git@github.com:Leonardobarony/meu-projeto.git (fetch)
# origin git@github.com:Leonardobarony/meu-projeto.git (push)
```

## 3. Fluxo Básico Diário

O fluxo diário serve para manter os 3 lugares sempre sincronizados: `git pull` → traz o GitHub pro PC | `git commit` → salva no Git local | `git push` → sobe pro GitHub

```
[Arquivos no PC] → [Git local] → [GitHub]
  você edita      git commit    git push
                  ← git pull ←
```

### Comando

### O que faz

|                                       |                                                                         |
|---------------------------------------|-------------------------------------------------------------------------|
| <code>git pull origin main</code>     | Baixa atualizações do GitHub — sempre o primeiro                        |
| <code>git status</code>               | Mostra o que mudou (vermelho = fora, verde = no stage)                  |
| <code>git add .</code>                | Adiciona tudo pro stage — ou <code>git add arquivo.py</code> para um só |
| <code>git commit -m "mensagem"</code> | Salva um ponto na história — seja descritivo                            |
| <code>git push origin main</code>     | Sobe os commits pro GitHub                                              |

```
bash

# Sequência do dia a dia
git pull origin main
# ... edita arquivos ...
git status
git add .
git commit -m "feat: nova funcionalidade"
git push origin main
```

Atalho — add + commit juntos (só arquivos já rastreados): `git commit -am "fix: correção rápida"`

## 4. Branches

Branch é uma linha do tempo paralela — você trabalha numa cópia sem mexer no código da `main`.

```
main:      A --- B --- C -----G (após merge)
              \               /
feature:    D --- E --- F
```

| Comando                                            | O que faz                      |
|----------------------------------------------------|--------------------------------|
| <code>git branch</code>                            | Lista branches locais          |
| <code>git branch -a</code>                         | Lista todas (incluindo GitHub) |
| <code>git checkout -b feature/nome</code>          | Cria e entra na branch         |
| <code>git checkout main</code>                     | Volta pra main                 |
| <code>git merge feature/nome</code>                | Mescla branch na atual         |
| <code>git branch -d feature/nome</code>            | Deleta local                   |
| <code>git push origin --delete feature/nome</code> | Deleta no GitHub               |

### Fluxo completo com branch

#### 1. No terminal — cria e sobe a branch:

```
bash

git checkout main
git pull origin main           # Atualiza main
git checkout -b feature/cadastro # Cria branch nova
# ... edita arquivos ...
git add .
git commit -m "feat: tela de cadastro"
git push -u origin feature/cadastro # Sobe pro GitHub
```

#### 2. No GitHub (manualmente no site):

- Abre o Pull Request

- Revisa o código
- Clica em **Merge**

### 3. No terminal — após o merge no GitHub:

```
bash

git checkout main
git pull origin main          # Baixa o merge que foi feito no GitHub
git branch -d feature/cadastro # Limpa a branch local do PC
```

### Conflito — como resolver

Acontece quando duas pessoas editam o mesmo trecho:

```
<<<<<< HEAD
    código da main
=====
    código da sua branch
>>>>>> feature/cadastro
```

1. Edite o arquivo manualmente — deixe como deve ficar
2. Remova as marcações <<<<, =====, >>>>
3. Salve e faça:

```
bash

git add .
git commit -m "fix: resolve conflito"
```

No VS Code aparecem botões visuais para aceitar um lado ou os dois — muito mais fácil.

## 5. Desfazer Coisas

### Ainda não fez git add

```
bash

git restore arquivo.py # Descarta mudança de um arquivo
git restore .          # Descarta tudo
```

### Já fez git add mas não commitou

```
bash
```

```
git restore --staged .      # Tira do stage (mudança continua no arquivo)
```

## Já commitou — desfazer o último commit

```
bash

# Mantém as mudanças nos arquivos (mais seguro)
git reset --soft HEAD~1

# Descarta tudo do último commit (cuidado!)
git reset --hard HEAD~1
```

## Já fez push — reverter com segurança

```
bash

git revert HEAD          # Cria um commit que desfaz o último
git push origin main     # Sobe a reversão
```

**revert** é o mais seguro quando já subiu pro GitHub, pois não reescreve o histórico.

---

## 6. Stash — Guardar Temporariamente

Útil quando precisa trocar de branch sem commitar o que está fazendo:

```
bash

git stash          # Guarda as mudanças
git stash list     # Lista o que está guardado
git stash pop      # Restaura o último stash
git stash drop     # Descarta o último stash
```

## Exemplo prático

```
bash
```

```
# Está trabalhando na feature/x e precisa urgente corrigir a main
git stash                # Guarda o trabalho
git checkout main
git pull origin main
# ... faz a correção ...
git commit -m "fix: correção urgente"
git push origin main
git checkout feature/x
git stash pop            # Restaura onde parou
```

## 7. Inspecionar o Histórico

```
bash

git log --oneline        # Histórico resumido
git log --oneline --graph # Com visual de branches
git diff                 # O que mudou antes do add
git diff --staged        # O que mudou depois do add
git show abc1234         # Ver detalhes de um commit
```

## 8. Resolver Conflitos

Acontece quando duas pessoas editam o mesmo trecho:

```
<<<<<< HEAD
    seu código (branch atual)
=====
    código que veio do merge
>>>>>> feature/nome
```

**Como resolver:**

1. Abra o arquivo e edite manualmente — deixe como deve ficar
2. Remova as marcações `<<<<`, `=====`, `>>>>`
3. Salve e faça:

```
bash

git add .
git commit -m "fix: resolve conflito"
```

---

## 🚩 9. Tags — Marcar Versões

```
bash

git tag v1.0.0           # Criar tag
git tag                  # Listar tags
git push origin v1.0.0   # Subir tag pro GitHub
git push origin --tags   # Subir todas as tags
```

---

## 🏷️ 10. Padrão de Mensagens de Commit

| Prefixo   | Uso                               |
|-----------|-----------------------------------|
| feat:     | Nova funcionalidade               |
| fix:      | Correção de bug                   |
| docs:     | Documentação                      |
| style:    | Formatação, sem mudança de lógica |
| refactor: | Refatoração de código             |
| test:     | Testes                            |
| chore:    | Tarefas de build, configs         |

### Exemplos:

```
bash

git commit -m "feat: adiciona autenticação OAuth"
git commit -m "fix: corrige erro no cálculo de datas"
git commit -m "refactor: simplifica query de vendas"
```

---

## Resumo de Emergência

| Situação                              | Comando                              |
|---------------------------------------|--------------------------------------|
| Errei antes do add                    | <code>git restore .</code>           |
| Errei depois do add                   | <code>git restore --staged .</code>  |
| Errei o commit (local)                | <code>git reset --soft HEAD~1</code> |
| Errei o commit (já no GitHub)         | <code>git revert HEAD</code>         |
| Preciso trocar de branch sem commitar | <code>git stash</code>               |
| Estou perdido                         | <code>git status</code>              |
|                                       |                                      |
|                                       |                                      |

Guia gerado em junho/2026 — [github.com/Leonardobarony](https://github.com/Leonardobarony)